
CS 61A STRUCTURE AND INTERPRETATION OF COMPUTER PROGRAMS MIDTERM 1

Fall 2026 Solutions

MONDAY, SEPTEMBER 14, 2026

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT Your @berkeley.edu Email: _____

PRINT Your Exam Room: _____

PRINT the Name of Person to your Left: _____

PRINT the Name of Person to your Right: _____

INSTRUCTIONS

You have **50 minutes** to complete the exam. There are **4 questions** and **6 pages** on this exam, including this cover page.

Question	1	2	3	4	Total
Points	6	4	4	6	20

- The exam is closed book, closed notes, closed computer, closed calculator, except one 8.5" × 11" page of your own creation and the provided Midterm 1 Study Guide.
 - Mark your answers on the exam itself in the spaces provided. We will not grade answers outside the designated answer spaces.
 - You **will not** be permitted to continue your exam after using the restroom unless you have a specific accommodation.
 - You may use built-in Python functions that do not require import, such as `pow`, `abs`, `bool`, `round`, `max`, and `min`.
 - You **may not** use example functions defined on your study guide unless a problem clearly states you can.
 - Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
 - You may not use `;` to place two statements on the same line.
-

Questions with **circular bubbles**: you may select only **1 choice**. Questions with **square boxes**: you may select **1 or more choices**.

☐ Unselected option (completely unfilled)

☐ You may select multiple squares

☒ Single option selected (completely filled)

☐ as long as they are completely filled

You must fill in the bubbles **completely**. Ticks, crosses, or other check marks will **not** receive credit.

HONOR CODE

"As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others."

SIGN Your Name: _____

1. [6.0 points] What Would Python Display?

Complete this What Would Python Display example by writing what the interactive interpreter would display on each blank line. For a function value, write FUNCTION. If an error occurs, write ERROR. Write your answers in the answer boxes. There are no score penalties in this question for extra/missing quotation marks or apostrophes.

```
>>> z = 'but'
>>> f = lambda g: g(z)
>>> z = 'so'
>>> x, y = print(1 or 2), (3 and 0)
(a) _____
>>> print(print(x, '=', y), '+', f(print), '= zero?')
(b) _____
(c) _____
(d) _____
```

[1.0 pt] Fill in blank (a).

1

[2.0 pts] Fill in blank (b).

None = 0

[1.0 pt] Fill in blank (c).

so

[2.0 pts] Fill in blank (d).

None + None = zero?

2. [4.0 points] New Print

Your friend hates that `print` returns `None`! They've tried to fix it...

In the large answer box on the right, write what is displayed by this code. It's recommended, but not required, that you draw an environment diagram in the space below. The answer is the same whether it is run in a file or in an interactive interpreter.

This code does not run forever or error. You may not need all of the frames in the diagram.

```
def new_print(print):
    """Return a better print that returns its argument"""
    def f(x):
        value = print(x)
        if value != None:
            return print('What?')
        return x
    return f

og = print
print = new_print(print)
print = new_print(print)
og('print returned', print(2))
```

Write your answer here:

```
2
What?
print returned What?
```

Global frame	

f1: _____ [parent=_____]	
Return Value	

f2: _____ [parent=_____]	
Return Value	

f3: _____ [parent=_____]	
Return Value	

f4: _____ [parent=_____]	
Return Value	

f5: _____ [parent=_____]	
Return Value	

3. [4.0 points] Long Odds

Definition. A *group* is a consecutive sequence of digits that are either all odd or all even.

Implement `num_long_groups`, which takes positive integers `n` and `k`. It returns the number of *groups* in `n` that have at least `k` digits. A group that is fully within another group is **not** counted. For example, 15732429999 has three groups: 1573, 242, and 9999. There are 3 groups with at least 3 digits, but only 2 groups with at least 4 digits, and 0 groups with at least 5 digits.

You **may not** write `str`, `len`, `if`, `else`, `[, ]`, 3, 4, 5, 6, 7, 8, or 9 in your answers.

```
def num_long_groups(n, k):  
    """Return the number of groups of consecutive digits of n that are either all odd or all even  
    and have at least k digits.
```

```
>>> num_long_groups(15732429999, 3)    # 1573 and 242 and 9999  
3  
>>> num_long_groups(15732429999, 4)    # 1573 and 9999  
2  
>>> num_long_groups(15732429999, 5)    # No group has 5 digits  
0  
>>> num_long_groups(15732429999, 1)    # All 3 groups have at least 1 digit  
3  
"""
```

```
assert n > 0 and k > 0, 'n and k must be positive.'
```

```
group_size, count = 0, 0
```

```
while n:
```

```
    group_size += 1
```

```
    if _____:  
        (a)
```

```
        count += 1
```

```
    if _____:  
        (b)
```

```
        group_size = 0
```

```
    n = n // 10
```

```
return count
```

[2.0 pts] Fill in blank (a).

- | | | | |
|---|---|--|--|
| ☐ <code>group_size < k</code> | ☐ <code>group_size <= k</code> | ☐ <code>group_size > k</code> | ☐ <code>group_size >= k</code> |
| ☒ <code>group_size == k</code> | ☐ <code>group_size != k</code> | ☐ <code>group_size % k == 0</code> | ☐ <code>group_size % k != 0</code> |

[2.0 pts] Fill in blank (b).

```
n % 2 != n // 10 % 2
```

4. [6.0 points] Space Between

You **may not** write `if`, `else`, `[, ]`, `for`, or `range` in your answers. You may use built-in functions: `min`, `max`, `pow`, `abs`, etc.

(Q4.1) Implement `gap`, which takes an integer `n` larger than 1 and a one-argument function `term`. It returns the largest absolute difference between any two consecutive terms in the sequence created by passing the numbers from 1 up to `n` (including `n`) to `term`.

```
def gap(n, term):
    """Return the largest absolute difference between consecutive terms of term(1), term(2), ..., term(n).

    >>> gap(5, lambda x: x * x)          # 1, 4, 9, 16, 25: 25 - 16
    9
    >>> gap(6, lambda x: x % 4)          # 1, 2, 3, 0, 1, 2: 3 - 0
    3
    >>> gap(4, lambda x: -x)              # -1, -2, -3, -4: every gap is 1
    1
    """
    k, largest = 1, 0

    while ____:
        (a)

        largest = ____
        (b)

        k = k + 1

    return largest
```

[1.0 pt] Fill in blank (a).

☐ `k < n - 1`

☒ `k < n`

☐ `k < n + 1`

☐ `k < n + 2`

[3.0 pts] Fill in blank (b).

```
max(largest, abs(term(k + 1) - term(k)))
```

(Q4.2) Implement `odd_gap`, which takes an integer `n` larger than 2 and a one-argument function `term`. It returns the largest absolute difference between any two consecutive terms in the sequence created by passing **only the odd** numbers from 1 up to `n` (including `n` if `n` is odd) to `term`. The `odd_gap` function calls the `gap` function.

```
def odd_gap(n, term):
    """Return the largest absolute difference between consecutive terms of term(1), term(3), term(5), ...

    >>> odd_gap(5, lambda x: x * x)      # term(1) = 1, term(3) = 9, term(5) = 25: 25 - 9
    16
    >>> odd_gap(8, lambda x: x % 5)      # term(1) = 1, term(3) = 3, term(5) = 0, term(7) = 2: 3 - 0
    3
    """
    return gap((n + 1) // 2, ____
    (c)
```

[2.0 pts] Fill in blank (c).

```
lambda k: term(2 * k - 1)
```

(Q4.3) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `make_gap`, which uses `gap` and the helper function `f` that is implemented below. `make_gap` takes a one-argument function `use` and returns a two-argument function. The returned function takes an integer `n` and a one-argument function `term`, and it returns the largest absolute difference between any two consecutive terms in the sequence created by passing to the `term` function only the numbers 1 up to `n` for which `use` returns a true value. Assume that `use` returns a true value for at least two of those numbers.

You **may not** write `for`, `:=`, `[, ]`, `if`, `else`, `and`, `or`, `map`, `filter`, or `range` in your solution. Use the features of Python that we have covered in the course so far.

Hint: Call `f` twice with different `go` and then arguments.

```
def f(use, go, then):
    """Helper for make_gap."""
    y, i = 0, 0
    while go(y, i):
        i += 1
        if use(i):
            y += 1
    return then(y, i)

def make_gap(use):
    """Return a gap function that only includes term(i) in the sequence when use(i) is a true value.

    >>> odd = lambda i: i % 2 == 1
    >>> odd_gap = make_gap(odd)          # this odd_gap has the same behavior as the one you defined before
    >>> odd_gap(8, lambda x: x % 5)     # 1, 3, 0, 2: 3 - 0
    3
    """
    return lambda n, term: gap(_____)
```

Fill in the blank in `make_gap`. If your answer is long, you may write it on multiple lines. You may write `λ` instead of `lambda`.

```
f(use, lambda y, i: i < n, lambda y, i: y), lambda k: term(f(use, lambda y, i: y < k, lambda y, i: i))
```